From Natural Language to Formal Proof: An Intersemiotic-Translation Lens on the Reliability Gap in LLM Theorem Proving

Anonymous Author
Challenge C2 Submission
October 8, 2026

Abstract

Translating a human-written mathematical proof into a formal language such as Lean or Coq is, in Roman Jakobson's classic typology, an *intersemiotic translation*: an interpretation of one sign system (informal mathematical English) by another (a typed calculus with no tolerance for implicature). We adopt this lens to re-examine a puzzling finding in neural theorem proving: state-of-the-art provers on the standard MINIF2F benchmark still reach only $\approx$73% pass@k, and DeepMind's silver-medal ALPHAPROOF at IMO 2024 relied on *manually translated* problem statements. We argue that end-to-end pass@k figures are misleading because they collapse four qualitatively distinct translation bottlenecks into a single number. We decompose the gap into four layers—(L1) theorem formalization, (L2) proof planning, (L3) low-level tactic detail, and (L4) search and repair—and map published systems onto them. Read through translation studies, these layers correspond to familiar failure types: cultural/register untranslatability (L1), loss of dynamic-equivalence cohesion (L2), collocational and grammatical errors (L3), and iterative revision cycles (L4). Two findings stand out. *First*, L4 has been aggressively pushed by tree-search methods (e.g., BFS-Prover at 72.95%) but L1 remains essentially unsolved. *Second*, a Dafny back-end solves 40.6% of MINIF2F-test with *empty* proofs, a symbolic baseline almost no neural-proving paper reports; the true LLM increment on the residual problems is closer to 54% than to 73%. We propose a layer-resolved reporting protocol and argue that the next breakthrough is more likely at L1/L2 than from further L4 search engineering.

1 Introduction

Large language models (LLMs) now write fluent natural-language (NL) mathematical reasoning, but such reasoning is unverifiable: a coherent chain-of-thought may contain an algebraic slip, a wrong lemma, or a fabricated citation, and the reader—human or automated—has no deterministic way to certify correctness Mao et al. [2024], Frieder et al. [2023], Collins et al. [2023]. Formal theorem proving (FTP) offers the opposite trade-off: languages such as Lean, Coq, Isabelle/HOL, and Dafny require every step to type-check, and the proof assistant itself becomes a deterministic auditor. If an LLM can emit a proof the assistant accepts, the result is, modulo the trusted kernel, *certified correct*.

This promise has driven a fast-moving sub-field. Yet the headline numbers are puzzling. On MINIF2F-test, the de facto cross-assistant benchmark, GPT-4 zero-shot reaches 25.4% Wang et al. [2024], fine-tuned open models reach 33.6% Wang et al. [2024], and ByteDance's BFS-Prover in 2025 sets the state of the art at 72.95% pass@k Xin et al. [2025]. Even DeepMind's silver-medal system at IMO 2024 combined ALPHAGEOMETRY2 Chervonyi et al. [2025] with ALPHAPROOF for algebra and number theory, and, as Jana et al. [2025] explicitly note, *problem statements were manually translated* before automated proof synthesis began. Why, after two years of rapid progress, does an IMO-level system still need a human in the loop at the very first step?

We argue that the community's metric—end-to-end pass@k—is hiding the answer. A 73% pass rate on MINIF2F could mean that systems fail at four equally sized 27% layers, or that they fail exclusively on one hard layer while the other three are nearly solved. These two diagnoses imply completely different research agendas.

We read this pipeline through a lens familiar in translation studies and intercultural communication: following Jakobson's tripartite typology, moving from informal mathematical English to a typed formal calculus is an *intersemiotic translation*—an interpretation of one sign system by another Jakobson [1959]. Viewed this way, the four bottlenecks below are not merely engineering problems; they map onto well-understood categories of translation failure: cultural/register untranslatability, loss of dynamic-equivalence cohesion, collocational and grammatical error, and iterative revision. This lens is not decorative: it explains why L1 (the most "translation-like" layer) is the least engineered, and why purely engineering fixes (L4 search) have hit a ceiling rather than cracking the problem.

Contributions.

1. We decompose the NL-to-FL proof gap into four layers: **L1** theorem formalization (NL statement → FL statement), **L2** proof planning (high-level strategy decomposition), **L3** low-level tactic detail (unification, typeclass inference, rewrite direction), and **L4** search and repair (state-space exploration, compiler-feedback repair).
2. We map twelve published systems onto this decomposition and show that current effort is heavily concentrated on L4, while L1 is almost untouched outside specialised neuro-symbolic engines.
3. We highlight a baseline that is rarely reported: Dafny's empty-proof automation solves 40.6% of MINIF2F-test Bakšys et al. [2025]. Against this baseline the true LLM increment on the residual problems is ≈54%, not 73%.
4. We propose a layer-resolved evaluation protocol that makes the next research agenda diagnosable.

2 Background

Proof assistants. Lean 4 has become the dominant platform for neural theorem proving, largely because of the MATHLIB mathematical library and the LeanDojo environment Song et al. [2024]. Coq and Isabelle/HOL have longer histories and larger software-verification user bases; Dafny is unusual in that it ships with automated back-ends (SMT solvers) that can discharge goals without any user-written tactic. We refer to recent surveys for the broader landscape Lu et al. [2023], Meadows and Freitas [2023].

Benchmarks. MINIF2F is a cross-assistant benchmark of 244 test and 244 validation undergraduate-level problems, formalised in Lean, Isabelle, HOL Light, and Metamath. PROOFNET, TRIGO Xiong et al. [2023], and the Open Bootstrapped Theorems (OBT) Wang et al. [2024] extend coverage to informal-proof alignment and to symbolic simplification. We focus on MINIF2F because it is the only benchmark for which nearly every system reports a comparable number.

Pipeline paradigms. Three paradigms dominate. *Whole-proof generation* prompts the LLM for a complete proof document in one shot (Baldur for Isabelle; the Coq whole-proof line studied by Prasad [2024]). *Tactic prediction* feeds the current proof state to the LLM and asks for the next tactic, iteratively; Lean Copilot Song et al. [2024] is the canonical framework. *Tree search* wraps tactic prediction with a search algorithm (MCTS, best-first search, or DPO-refined expansion) that explores multiple proof states; BFS-Prover Xin et al. [2025] is the current standard-bearer.

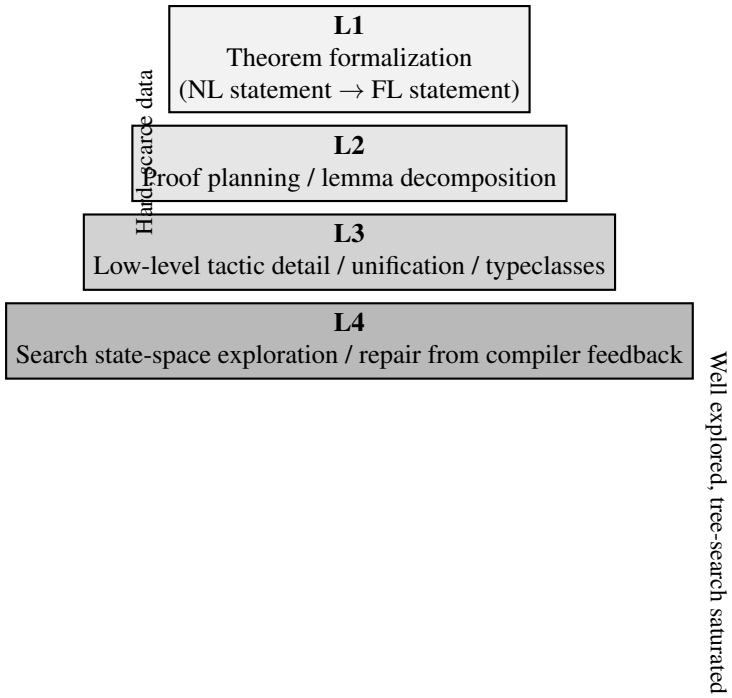


Figure 1: The four-layer decomposition of the NL-to-FL proof gap. Failure at a higher layer forces a human-in-the-loop step; failure at L4 is the layer most systems currently engineer.

3 The Four-Layer Gap

We posit that an end-to-end NL-to-FL proof pipeline can fail at four qualitatively distinct stages. Figure 1 summarises the decomposition; we discuss each layer with evidence from published error analyses.

3.1 L1: Theorem Formalization Gap

Input: a problem written for humans (e.g., an IMO statement in English). *Output:* a well-typed FL theorem that captures exactly the intended mathematical content, including implicit domains, variable types, and definitions drawn from the right library.

This layer is the least studied. Jana et al. [2025] explicitly motivate ProofBridge by observing that AlphaProof's silver-medal IMO result required human translation of problem statements. The challenge is not mere parsing: an NL statement like "f is continuous" may require choosing between `ContinuousMap`, `ContinuousOn`, and the topological definition depending on context, and the wrong choice makes downstream proof search fail for reasons that have nothing to do with the proof itself.

ProofBridge attacks L1 with a joint NL–FL embedding and retrieval- augmented fine-tuning, reporting a $+31.14\%$ semantic-correctness gain over a Kimina-Prover baseline on their curated miniF2F-Test-PF split Jana et al. [2025]. AlphaGeometry2 Chervonyi et al. [2025] covers 88% of IMO 2000–2024 geometry statements with its own controlled input language, an extension of the original AlphaGeometry's neuro-symbolic pipeline Trinh et al. [2023]; but this coverage is achieved by restricting the input language—precisely the manual formalization step we identify as L1.

Translational reading. This layer is closest to what translation studies call *cultural untranslatability*: an NL statement draws on shared presuppositions of the mathematics discourse community Swales [1990]—

what "continuous" means in context, which library definition is intended—that have no one-to-one formal counterpart. Venuti's notion of *domestication versus foreignization* Venuti [1995] captures the design choice: ALPHAGEOMETRY2 "domesticates" by restricting the input language, while ProofBridge attempts "foreignization" by accepting raw NL. The modest +31% semantic-correctness gain suggests that foreignization is still brittle.

3.2 L2: Proof Planning Gap

Input: a formal theorem statement. *Output:* a sound high-level decomposition into lemmas or sub-goals, without filling in the low-level justifications.

Evidence that LLMs are weak at this layer comes from controlled synthetic experiments. Saparov and He [2023] parse generated chain-of-thought into symbolic proofs in ProntoQA and find that models produce *individually correct deduction steps* but fail at *proof planning*: when multiple valid deductions are available, they do not systematically explore alternatives and converge on shallow, often dead-end paths. Opedal et al. [2024] extend this finding by generating problems with arbitrarily deep/wide proof trees and showing performance degrades markedly as proof depth grows.

BC-Prover He et al. [2024] is a direct response: it introduces "pseudo-steps" that decompose the proof into backward-chained sub-goals *before* generating formal tactics, and reports significant MINIF2F gains. The very need for such a module confirms that vanilla tactic prediction leaves L2 unaddressed.

Translational reading. In Nida's classic distinction Nida [1964], NL proofs strive for *dynamic equivalence*: a reader understands the intended argument via connectives such as "it remains to show that" that elide routine steps. Formal proofs instead require *formal equivalence* at every step. The L2 failure is precisely the loss of cohesion when dynamic-equivalence connectives are stripped: the model knows the destination but cannot map NL discourse markers onto a valid sub-goal tree. This is why BC-Prover's "pseudo-steps" work—they reintroduce an explicit discourse planner that the LLM otherwise omits.

3.3 L3: Low-Level Tactic Detail Gap

Input: a proof state and a desired high-level step. *Output:* a well-typed tactic invocation with the right arguments, typeclass instances, rewrites, and implicit parameters.

This is the layer where error analysis is richest. Lu et al. [2024] analysed 520 GPT-3.5 proof-generation errors in Coq and found a strikingly consistent pattern: the model *identified the correct high-level structure* of the proof but failed on low-level details (wrong unification, missing arguments, incorrect rewrite direction). Their PALM system generates a proof then applies targeted symbolic repairs to low-level failures, improving over baselines by 76.5%–180.4% across more than 10K theorems.

APRIL Wang et al. [2026] operationalises the same insight for Lean: it constructs a 260K-tuple dataset of erroneous proofs paired with compiler diagnostics and aligned repairs, and shows that a fine-tuned 4B model beats stronger open-source baselines on single-shot repair. Both systems treat L3 as a feedback-driven repair problem rather than a one-shot generation problem.

Translational reading. These are *collocational and grammatical errors*: the equivalent of a translator who chooses the right word but pairs it with the wrong preposition, tense, or case. The LLM "speaks" the formal language at the level of register Swales [1990] but violates its selectional restrictions—which arguments a tactic expects, in which order, with which implicit instance. APRIL's compiler-feedback loop is analogous to a translator revising against native-speaker annotations: a correction mechanism that presupposes the error has already happened, rather than preventing it.

System	Year	L1	L2	L3	L4	MINIF2F-test
GPT-4 zero-shot (FL theorem)	2023	–	∼	∼	–	25.4%
TheoremLlama Wang et al. [2024]	2024	bootstrapped	∼	✓	–	33.6%
Lean Copilot Song et al. [2024]	2024	–	✓	✓	–	n/a (step-level
PALM Lu et al. [2024]	2024	–	∼	✓ (repair)	∼	+76–180%†
BC-Prover He et al. [2024]	2024	–	✓ (pseudo-steps)	∼	✓	SOTA at time
BFS-Prover Xin et al. [2025]	2025	–	∼	∼	✓ (DPO+BFS)	**72.95%**
ProofBridge Jana et al. [2025]	2025	✓ (joint emb.)	∼	✓	✓ (repair)	+31.1% SC‡
Dafny empty-proof Bakšys et al. [2025]	2025	n/a	n/a	n/a	SMT	**40.6%**

† Relative improvement over baselines on a >10K-theorem Coq corpus, not MINIF2F.

‡ Semantic-correctness gain on the curated MINIF2F-Test-PF split.

Table 1: Systems mapped onto the four layers. L4 is the most densely addressed layer; L1 is addressed only by ProofBridge and the geometry-specific ALPHAGEOMETRY2 input language. The bottom row is the symbolic baseline that almost no LTP paper reports.

3.4 L4: Search and Repair Gap

Input: an initial proof state. *Output:* a path to a closed state, allowing backtracking and intermediate repairs.

This is the layer that has seen the most engineering investment. BFS-Prover Xin et al. [2025] reports 72.95% on MINIF2F-test using best-first tree search with DPO on compiler-error-annotated state–tactic pairs. The Coq whole-proof line of Prasad [2024] lifts GPT-4 from 66.44% to 79.42% by recursively extracting lemmas at failure points—a search strategy. Lean Copilot Song et al. [2024] automates 74.2% of proof steps in the MATHEMATICS IN LEAN textbook, an 85% gain over rule-based Aesop.

We read these numbers as evidence that L4 is approaching a local ceiling given current LLM tactic-prediction quality: further gains from better search alone appear to be incremental, not transformational.

Translational reading. This layer is the *revision cycle* of professional translation: draft, consult parallel texts, backtrack, try again. BFS-Prover and the Coq lemma-extraction loop are industrial-scale versions of the translator's draft-and-revise workflow. That L4 is the most engineered layer makes sense—it is the one both translators and their computer-aided translation tools already know how to optimise. It is also, as the 72.95% ceiling suggests, the layer where further investment yields diminishing returns: better search cannot fix a proof whose L1 formalization was wrong or whose L2 plan was ill-chosen.

4 Evidence Mapping

Table 1 places representative systems on the four layers. "✓" denotes an explicit module addressing that layer; "∼" denotes incidental or partial treatment; "–" denotes no targeted mechanism.

Two observations follow.

First, L4 is saturated; L1 is open. Seven of eight neural systems have an L4 component, but only ProofBridge explicitly addresses L1, and even ProofBridge assumes the NL theorem is available and focuses on translating it. No system in Table 1 fully automates the path from an arbitrary olympiad NL statement to a formalised theorem *and* a verified proof. This is consistent with the manual-translation caveat attached to ALPHAPROOF.

Second, improvements are cumulative, not compositional. BFS-Prover's 72.95% is built on better L4 search but does not fix L3 root causes; PALM's 76–180% L3 repair does not ship with an L2 planner. A system that stacked L1 (ProofBridge) + L2 (BC-Prover pseudo-steps) + L3 (PALM/APRIL repair) + L4

(BFS search) has not, to our knowledge, been published.

5 The Baseline Nobody Reports

MINIF2F-Dafny Bakšys et al. [2025] reports that Dafny's built-in SMT automation discharges 99/244 (40.6%) of the *test* set and 109/244 (44.7%) of the validation set with *empty proofs*—no LLM, no tactic, no hint. For the residual problems, the best off-the-shelf LLM reaches 55.7% pass@4 with iterative error correction.

This reframes the headline number. Let $p_{\mathrm{LLM}} = 0.7295$ be BFS-Prover's pass rate and $p_{\mathrm{SMT}} = 0.406$ be the empty-proof Dafny rate. The LLM's *incremental* contribution on the problems that symbolic automation cannot solve is

$$\Delta_{\mathrm{LLM}} = \frac{p_{\mathrm{LLM}} - p_{\mathrm{SMT}}}{1 - p_{\mathrm{SMT}}} = \frac{0.7295 - 0.406}{1 - 0.406} \approx 0.543. \tag{1}$$

In other words, the LLM closes about 54% of the residual gap, not 73% of the total benchmark. We do not claim that BFS-Prover and Dafny are directly comparable (they target different proof assistants and different automation layers), but the methodological point stands: *any neural-proving result should be reported alongside the performance of a symbolic-only baseline on the same problems*. Without this, a 73% figure can look twice as impressive as the underlying LLM increment justifies.

6 Toward Layer-Resolved Evaluation

We propose that future neural theorem-proving papers report, in addition to end-to-end pass@k, the following per-layer quantities:

- p_1: fraction of NL statements automatically formalised with *semantic equivalence* verified by proving the bidirectional implication of the NL-intended and FL statements (the metric ProofBridge already uses).
- p_2: given a correctly formalised theorem, fraction for which the system proposes a *sound high-level decomposition* judged by an oracle or by whether the sub-goals are discharged.
- p_3: given a correct high-level step, fraction whose first tactic attempt type-checks (L3 hit rate).
- p_4: given a failed first attempt, fraction eventually discharged through search/repair within a fixed budget.

These are not independent probabilities in a multiplicative sense (repair can bootstrap back across layers), but reporting them separately makes the layer at which a system actually improves visible. A system that claims "73% on MINIF2F" but reports $p_1 = 0.1$, $p_2 = 0.4$, $p_3 = 0.5$, $p_4 = 0.8$ is diagnosing itself: the next gain must come from L1 or L2, not from more search.

7 Discussion and Limitations

We synthesise existing evidence; we do not introduce a new system or run new experiments. Our layer definitions are post hoc and our mapping of systems onto them is based on published descriptions, not on re-implementation. Three caveats deserve emphasis.

Benchmark skew. MINIF2F is predominantly undergraduate algebra and analysis. On graduate-level material, MATHLIB coverage and the L1 formalization burden grow sharply; the layer distribution we report may shift. **Assistant ecology skew.** Lean's dominance in neural proving reflects MATHLIB's training-data advantage, not an intrinsic property of the LTP problem. **Repair is not free.** Systems that achieve high

end-to-end numbers via multi-attempt search or repair (BFS-Prover, PALM) consume orders of magnitude more compute than a single pass; the pass@k figure hides this cost.

Why a translation-studies lens? Readers from a formal-methods background may wonder why we invoke Jakobson, Nida, and Swales in a paper about theorem provers. The reason is diagnostic: the layers we have labelled L1–L4 are not equally amenable to engineering, and the translation tradition has long experience distinguishing failure types that are *structurally* different. L1 (cultural untranslatability) cannot be fixed by faster search; L3 (collocational error) can; L2 (loss of dynamic equivalence) requires a discourse planner, not a larger model. Viewing LLM theorem proving as intersemiotic translation Jakobson [1959] also highlights a cross-cultural dimension: NL mathematical prose is itself a high-register, low-redundancy discourse Swales [1990], Hall [1976], and the gap to a formal calculus is an extreme case of translating between discourse communities. We are not proposing that theorem provers be trained on translation theory; we are proposing that their failure modes be *classified* with the same care that translation scholars apply to literary translation, because the current engineering practice of reporting only pass@k conflates four different failure types.

8 Conclusion

Formal theorem proving is the rare setting where an LLM's output can be deterministically certified, yet the field's headline numbers obscure where the remaining difficulty lies. Reading the pipeline as Jakobson's intersemiotic translation Jakobson [1959] has allowed us to decompose the NL-to-FL proof gap into four qualitatively distinct layers and to show that published effort is concentrated on L4 (search), while L1 (automated formalization of problem statements) and L2 (proof planning) remain structurally under-addressed. A Dafny empty-proof baseline of 40.6% further suggests that the true LLM increment on MINIF2F is closer to 54% than to the commonly quoted 73%. We recommend layer-resolved reporting so that the next breakthrough—likely at L1 or L2 rather than at further L4 search engineering—can be diagnosed as it happens.

References

Mantas Bakšys, Stefan Zetzsche, and Olivier Bouissou. miniF2F-Dafny: Llm-guided mathematical theorem proving via auto-active verification. *arXiv preprint arXiv:2512.10187*, 2025. URL https://arxiv.org/abs/2512.10187.

Yuri Chervonyi, Trieu H. Trinh, Miroslav Olšák, Xiaomeng Yang, Hoang H. Nguyen, Marcelo Menegali, Junehyuk Jung, Junsu Kim, Vikas Verma, Quoc V. Le, and Thang Luong. Gold-medalist performance in solving olympiad geometry with AlphaGeometry2. *Journal of Machine Learning Research (JMLR)*, 26 (25):1–50, 2025. URL https://www.jmlr.org/papers/v26/25-1654.html.

Katherine M. Collins, Albert Q. Jiang, Simon Frieder, Lionel Wong, Miri Zilka, Umang Bhatt, Thomas Lukasiewicz, Yuhuai Wu, Joshua B. Tenenbaum, William Hart, Timothy Gowers, Wenda Li, Adrian Weller, and Mateja Jamnik. Evaluating language models for mathematics through interactions. *arXiv preprint arXiv:2306.01694*, 2023. URL https://arxiv.org/abs/2306.01694.

Simon Frieder, Luca Pinchetti, Alexis Chevalier, Ryan-Rhys Griffiths, Tommaso Salvatori, Thomas Lukasiewicz, Philipp Petersen, and Julius Berner. Mathematical capabilities of ChatGPT. In *Advances in Neural Information Processing Systems (NeurIPS), Track on Datasets and Benchmarks*, 2023. URL https://papers.nips.cc/paper_files/paper/2023/hash/

58168e8a92994655d6da3939e7cc0918-Abstract-Datasets_and_Benchmarks.html.

Edward T. Hall. *Beyond Culture*. Anchor Books, Garden City, NY, 1976.

Yuhang He, Jihai Zhang, Jianzhu Bao, Fangquan Lin, Cheng Yang, Bing Qin, Ruifeng Xu, and Wotao Yin. BC-Prover: Backward chaining prover for formal theorem proving. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024. URL https://aclanthology.org/2024.emnlp-main.180/.

Roman Jakobson. *On Linguistic Aspects of Translation*, pages 232–239. Harvard University Press, Cambridge, MA, 1959.

Prithwish Jana, Kaan Kale, Ahmet Ege Tanriverdi, Cruise Song, Sriram Vishwanath, and Vijay Ganesh. Proofbridge: Auto-formalization of natural language proofs in lean via joint embeddings. *arXiv preprint arXiv:2510.15681*, 2025. URL https://arxiv.org/abs/2510.15681.

Minghai Lu, Benjamin Delaware, and Tianyi Zhang. Proof automation with large language models. *arXiv preprint arXiv:2409.14274*, 2024. URL https://arxiv.org/abs/2409.14274.

Pan Lu, Liang Qiu, Wenhao Yu, Sean Welleck, and Kai-Wei Chang. A survey of deep learning for mathematical reasoning. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023. URL https://aclanthology.org/2023.acl-long.817/.

Yujun Mao, Yoon Kim, and Yilun Zhou. CHAMP: A competition-level dataset for fine-grained analyses of LLMs' mathematical reasoning capabilities. In *Findings of the Association for Computational Linguistics: ACL*, 2024. URL https://aclanthology.org/2024.findings-acl.785/.

Jordan Meadows and André Freitas. Introduction to mathematical language processing: Informal proofs, word problems, and supporting tasks. *Transactions of the Association for Computational Linguistics (TACL)*, 11:1182–1200, 2023. URL https://aclanthology.org/2023.tacl-1.66/.

Eugene A. Nida. *Toward a Science of Translating: With Special Reference to Principles and Procedures Involved in Bible Translating*. E. J. Brill, Leiden, 1964.

Andreas Opedal, Haruki Shirakami, Bernhard Schölkopf, Abulhair Saparov, and Mrinmaya Sachan. MathGAP: Out-of-distribution evaluation on problems with arbitrarily complex proofs. *arXiv preprint arXiv:2410.13502*, 2024. URL https://arxiv.org/abs/2410.13502.

Tarun Prasad. *Guided Proof Search Using Large Language Models and Lemma Extraction in Coq*. PhD thesis, Harvard University, School of Engineering and Applied Sciences, 2024. URL https://dash.harvard.edu/handle/1/37370663.

Abulhair Saparov and He He. Language models are greedy reasoners: A systematic formal analysis of chain-of-thought. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2210.01240.

Peiyang Song, Kaiyu Yang, and Anima Anandkumar. Lean Copilot: Large language models as copilots for theorem proving in Lean. In *Proceedings of Machine Learning Research (PMLR)*, 2024. URL https://arxiv.org/abs/2404.12534.

John M. Swales. *Genre Analysis: English in Academic and Research Settings*. Cambridge University Press, Cambridge, 1990.

Trieu H. Trinh, Yuhuai Wu, Quoc V. Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 622:716–721, 2023. URL https://www.nature.com/articles/s41586-023-06747-5.

Lawrence Venuti. *The Translator's Invisibility: A History of Translation*. Routledge, London, 1995.

Ruida Wang, Jipeng Zhang, Yizhen Jia, Rui Pan, Shizhe Diao, Renjie Pi, and Tong Zhang. TheoremLlama: Transforming general-purpose LLMs into Lean4 experts. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024. URL https://aclanthology.org/2024.emnlp-main.667/.

Yiran Wang, Simon Chess, Daniel Lee, Siyuan Ge, Ajit Mallavarapu, and Vasily Ilin. Learning to repair Lean proofs from compiler feedback (APRIL). *arXiv preprint arXiv:2602.02990*, 2026. URL https://arxiv.org/abs/2602.02990.

Ran Xin, Chenguang Xi, Jie Yang, Feng Chen, Hang Wu, Xia Xiao, Yifan Sun, Shen Zheng, and Kai Shen. BFS-Prover: Scalable best-first tree search for LLM-based automatic theorem proving. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025. URL https://aclanthology.org/2025.acl-long.1565/.

Jing Xiong, Jianhao Shen, Ye Yuan, Haiming Wang, Yichun Yin, Zhengying Liu, Lin Li, Zhijiang Guo, Qingxing Cao, Yinya Huang, Chuanyang Zheng, Xiaodan Liang, Ming Zhang, and Qun Liu. TRIGO: Benchmarking formal mathematical proof reduction for generative language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. URL https://aclanthology.org/2023.emnlp-main.711/.